Codul analizei MQTTset

Scripturile care reproduc toate cifrele și graficele din document. Repo: https://github.com/Darius15222/mqtt-attack-analysis (folderul cod/); site: https://darius15222.github.io/mqtt-attack-analysis/

Rulare: `python3 -m venv venv && venv/bin/pip install pandas scipy statsmodels mlxtend matplotlib`, apoi `venv/bin/python 01_cap1-7_statistici_corelatii.py mqttdataset_reduced.csv.zip`. Arborii: `Rscript 09_cap10_arbori_ctree.R mqttdataset_reduced.csv.zip arbori` (pachetul R partykit).

1. `01_cap1-7_statistici_corelatii.py`
2. `02_cap7-9_regresii_logistica.py`
3. `03_cap9_probabilitati.py`
4. `04_cap9_firth.py`
5. `05_cap8-9_cook_durbin_watson.py`
6. `06_cap11_reguli_asociere.py`
7. `07_grafice_corelatie.py`
8. `08_grafice_restul.py`
9. `09_cap10_arbori_ctree.R`
10. `10_grafice_arbori.py`

01_cap1-7_statistici_corelatii.py

```
import sys, pandas as pd, numpy as np
from scipy import stats
d = pd.read_csv(sys.argv[1], low_memory=False)
print(d.shape); print(d.target.value_counts(), (d.target.value_counts(normalize=True)*100).round(2))
const=[c for c in d if d[c].nunique()==1]; print("const",len(const),const)
for c in ["tcp.len","tcp.time_delta","mqtt.len","mqtt.kalive"]:
    x=d[c]; print(c, "mean",round(x.mean(),2),"std",round(x.std(),2),"med",x.median(),"q",x.quantile([.25,.75]).tolist(),"skew",round(x.skew(),2),"kurt",round(x.kurt(),1))
    q1,q3=x.quantile([.25,.75]); hi=q3+1.5*(q3-q1); o=(x<q1-1.5*(q3-q1))|(x>hi)  # valori extreme, regula IQR
    print("  IQR: peste",hi,"n",o.sum(),round(o.mean()*100,2),"%", d.target[o].value_counts(normalize=True).round(3).to_dict())
for c in ["tcp.flags","mqtt.hdrflags","mqtt.msgtype","mqtt.conflags","mqtt.protoname","mqtt.conack.flags","mqtt.msg"]:
    vc=d[c].astype(str).value_counts(normalize=True); print(c, vc.index[0], round(vc.iloc[0]*100,2))
for c in ["mqtt.dupflag","mqtt.retain","mqtt.conflag.cleansess","mqtt.conflag.passwd","mqtt.conflag.uname","mqtt.proto_len","mqtt.ver","mqtt.conack.val","mqtt.qos"]:
    print(c, "nonzero%", round((d[c]!=0).mean()*100,2), "count", (d[c]!=0).sum(), d[c].unique()[:6])
# per class
leg=d[d.target=="legitimate"]
print("leg tcp.len med q", leg["tcp.len"].median(), leg["tcp.len"].quantile([.25,.75]).tolist(), "mean", round(leg["tcp.len"].mean(),2), "zero%", round((leg["tcp.len"]==0).mean()*100,1), "10-14%", round(leg["tcp.len"].between(10,14).mean()*100,1))
for c in ["tcp.time_delta","tcp.len","mqtt.len"]:
    print(c, {k:(g.median(), g.quantile(.25), g.quantile(.75)) for k,g in d.groupby("target")[c]})
    print(" KW H", stats.kruskal(*[g.values for _,g in d.groupby("target")[c]]))
for c in ["tcp.flags","mqtt.hdrflags","mqtt.msgtype"]:
    for k,g in d.groupby("target"):
        vc=g[c].astype(str).value_counts(normalize=True); print(" ",c,k,vc.index[0],round(vc.iloc[0]*100,2))
for c in ["mqtt.dupflag","mqtt.retain","mqtt.conflag.cleansess","mqtt.conflag.passwd","mqtt.conflag.uname","mqtt.conack.val","mqtt.qos"]:
    print(c, (d.groupby("target")[c].apply(lambda s:(s!=0).mean()*100)).round(2).to_dict())
print("kalive slowite 65535%", round((d[d.target=="slowite"]["mqtt.kalive"]==65535).mean()*100,2), d["mqtt.kalive"].unique()[:10])
O=pd.crosstab(d["tcp.flags"],d.target); print(O); ch=stats.chi2_contingency(O); print("chi2",ch[0],ch[2])
E=ch[3]; print("E FIN,ACK bf", E[O.index.get_loc("0x00000011"), O.columns.get_loc("bruteforce")])
O2=pd.crosstab(d["mqtt.conack.val"],d.target); c2=stats.chi2_contingency(O2); print(O2, "chi2",c2[0]); print("E", c2[3])
# MW flood vs normal time_delta
n=leg["tcp.time_delta"]; f=d[d.target=="flood"]["tcp.time_delta"]
u=stats.mannwhitneyu(n,f); print("MW",u, len(n)*len(f))
u2=stats.mannwhitneyu(f,n); print("MW flipped",u2)
# rows
for k in d.target.unique(): print(k, list(d.index[d.target==k][:2]+1))
print(d.loc[[0,4,1,46,36,6,1420],["tcp.flags","tcp.len","mqtt.len","target"]])
# Spearman/Pearson
r13=["tcp.time_delta","tcp.len","mqtt.len","mqtt.kalive","mqtt.qos","mqtt.dupflag","mqtt.retain","mqtt.conflag.cleansess","mqtt.conflag.passwd","mqtt.conflag.uname","mqtt.proto_len","mqtt.ver","mqtt.conack.val"]
sp=d[r13].corr("spearman"); print(sp.round(2))
d["is_attack"]=(d.target!="legitimate").astype(int)
for a,b in [("tcp.len","mqtt.len"),("tcp.time_delta","tcp.len"),("tcp.time_delta","mqtt.len"),("tcp.len","is_attack"),("mqtt.len","is_attack"),("tcp.time_delta","is_attack"),("mqtt.qos","is_attack"),("mqtt.dupflag","is_attack")]:
    print(a,b, round(d[a].corr(d[b]),3), round(d[a].corr(d[b],method="spearman"),3))
print("conn nonzero rows", (d["mqtt.ver"]!=0).sum(), ((d["mqtt.ver"]!=0)==(d["mqtt.kalive"]!=0)).all())
q1=d[d["mqtt.qos"]==1]; print("qos1",len(q1), q1.target.unique(), q1["mqtt.len"].min(), q1["mqtt.len"].max(), d[d["mqtt.qos"]==0]["mqtt.len"].mean())
print("dup1 subset qos1", ((d["mqtt.dupflag"]==1)&(d["mqtt.qos"]!=1)).sum())
for k,g in d.groupby("target"):
    print(k, "tcp-mqtt spearman", round(g["tcp.len"].corr(g["mqtt.len"],method="spearman"),2), "nconst", sum(g[c].nunique()==1 for c in r13), [c for c in r13 if g[c].nunique()==1])
print("tcp.len max flood", d[d.target=="flood"]["tcp.len"].max(), "mqtt.len flood max", d[d.target=="flood"]["mqtt.len"].max())
x=d["tcp.len"];y=d["mqtt.len"]; print("sxy",((x-x.mean())*(y-y.mean())).sum(),"sxx",((x-x.mean())**2).sum(),"syy",((y-y.mean())**2).sum())
# example table 4 attack examples
att=d[d.is_attack==1]
for c in ["tcp.flags","tcp.time_delta","tcp.len","mqtt.hdrflags","mqtt.msgtype","mqtt.len","mqtt.qos","mqtt.dupflag","mqtt.kalive"]:
    s=d[c].astype(str); share=d.groupby(s)["is_attack"].mean(); ok=share[share>=0.99].index
    vc=att[c].astype(str).value_counts(); vc=vc[vc.index.isin(ok)]
    nv=leg[c].astype(str).value_counts(normalize=True)
```

```
55     print("T4",c,"normal",nv.index[0],round(nv.iloc[0]*100,1),"attack", (vc.index[0], round(vc.iloc[0]/len(at
t)*100,1)) if len(vc) else None)
```

02_cap7-9_regresii_logistica.py

```
import sys, pandas as pd, numpy as np, statsmodels.api as sm
from statsmodels.stats.outliers_influence import variance_inflation_factor as vif
from scipy import stats
d = pd.read_csv(sys.argv[1], low_memory=False); d["is_attack"]=(d.target!="legitimate").astype(int)
r13=["tcp.time_delta","tcp.len","mqtt.len","mqtt.kalive","mqtt.qos","mqtt.dupflag","mqtt.retain","mqtt.conflag.cleansess","mqtt.conflag.passwd","mqtt.conflag.uname","mqtt.proto_len","mqtt.ver","mqtt.conack.val"]
pd.set_option("display.width",250); sp=d[r13].corr("spearman"); print(sp.round(2).to_string())
for k,g in d.groupby("target"):
    c=[x for x in r13 if g[x].nunique()>1]; m=g[c].corr("spearman").values; iu=np.triu_indices(len(c),1); print(k,"pairs",len(iu[0]),"r>=0.995",(m[iu]>=0.995).sum())
print("pearson qos-mqtt.len", d["mqtt.qos"].corr(d["mqtt.len"]))
f=d[d.target=="flood"]; print("flood mqtt.len when tcp.len>1460", f[f["tcp.len"]>1460]["mqtt.len"].describe().to_dict()); print("flood ACK empty", ((f["tcp.flags"]=="0x00000010")&(f["tcp.len"]==0)).sum(), "tcp.len>1460 all", (d["tcp.len"]>1460).sum(), d[d["tcp.len"]>1460].target.unique())
print("legit tcp.len max", d[d.target=="legitimate"]["tcp.len"].max(), "time_delta max", d["tcp.time_delta"].max(), (d["tcp.time_delta"]>50).sum())
X=sm.add_constant(d[["tcp.len"]]); m1=sm.OLS(d["mqtt.len"],X).fit(); print("R1",m1.params.round(4).to_dict(), round(m1.rsquared,4), "ssr",m1.ssr, "res skew/kurt", round(stats.skew(m1.resid),2), round(stats.kurtosis(m1.resid),1))
L=np.log1p(d[["tcp.len","mqtt.len"]]); m1l=sm.OLS(L["mqtt.len"],sm.add_constant(L["tcp.len"])).fit(); print("R1log",m1l.params.round(3).to_dict(), round(m1l.rsquared,3), round(stats.skew(m1l.resid),2), round(stats.kurtosis(m1l.resid),1))
m2=sm.OLS(d["tcp.time_delta"],sm.add_constant(d[["tcp.len","mqtt.len"]])).fit(); print("R2",m2.params.to_dict(), m2.pvalues.to_dict(), m2.rsquared, stats.skew(m2.resid), stats.kurtosis(m2.resid))
for k,g in d.groupby("target"):
    a=sm.OLS(g["mqtt.len"],sm.add_constant(g["tcp.len"])).fit(); gl=np.log1p(g[["tcp.len","mqtt.len"]]); b=sm.OLS(gl["mqtt.len"],sm.add_constant(gl["tcp.len"])).fit()
    print("R3",k,len(g),a.params.round(4).tolist(), round(a.rsquared,3), round(b.rsquared,3))
P=["tcp.len","mqtt.len","tcp.time_delta","mqtt.qos","mqtt.dupflag"]; Xv=sm.add_constant(d[P]).values
print("VIF5",[round(vif(Xv,i),2) for i in range(1,6)]); X3=sm.add_constant(d[["tcp.len","tcp.time_delta","mqtt.len"]]).values; print("VIF3",[round(vif(X3,i),2) for i in range(1,4)])
g6=sm.GLM(d.is_attack,sm.add_constant(d[["tcp.len","tcp.time_delta","mqtt.len"]]),family=sm.families.Binomial()).fit()
print("LOG6",g6.params.round(4).to_dict(), g6.bse.round(4).to_dict(), g6.tvalues.round(2).to_dict(), g6.null_deviance, g6.deviance, g6.aic, "iters", g6.fit_history["iteration"])
for it in [10,15,20,25,30]:
    gA=sm.GLM(d.is_attack,sm.add_constant(d[P]),family=sm.families.Binomial()).fit(maxiter=it, tol=1e-30)
    print("LOG5 it",it, gA.params.round(3).to_dict(), gA.bse.round(1).to_dict(), round(gA.deviance,1))
pr=pd.Series(g6.predict(),index=d.index);
for k in d.target.unique():
    s=pr[d.target==k]; print("prob",k,"max",round(s.max(),3),"med",round(s.median(),3),">0.9",round((s>0.9).mean(),3),"~0.42",round((np.abs(s-0.4235)<0.005).mean(),3))
att=d[d.is_attack==1]; leg=d[d.is_attack==0]
for c in ["tcp.time_delta","tcp.len","mqtt.len"]: print("9.2",c,att[c].median(),att[c].quantile([.25,.75]).tolist())
print("msgtype by class", pd.crosstab(d["mqtt.msgtype"],d.target).to_string())
```

03_cap9_probabilitati.py

```
import sys, pandas as pd, numpy as np, statsmodels.api as sm
d = pd.read_csv(sys.argv[1], low_memory=False); d["is_attack"]=(d.target!="legitimate").astype(int)
g6=sm.GLM(d.is_attack,sm.add_constant(d[["tcp.len","tcp.time_delta","mqtt.len"]]),family=sm.families.Binomial()).fit()
pr=pd.Series(np.asarray(g6.predict()),index=d.index)
for k in d.target.unique():
    s=pr[d.target==k]; print("prob",k,"max",round(s.max(),3),"med",round(s.median(),3),">0.9",round((s>0.9).mean(),3),"~0.42",round((np.abs(s-0.4235)<0.005).mean(),3))
b=g6.params; print("curve d=0 typical", 1/(1+np.exp(-(b.const+b["tcp.len"]*13+b["mqtt.len"]*11))))
att=d[d.is_attack==1]
for c in ["tcp.time_delta","tcp.len","mqtt.len"]: print("9.2",c,att[c].median(),att[c].quantile([.25,.75]).tolist())
print(pd.crosstab(d["mqtt.msgtype"],d.target).to_string())
dos=d[(d.target=="dos")&(d["mqtt.msgtype"]==3)&(d["tcp.flags"]=="0x00000010")]; print("dos publish ACK", len(dos), dos["tcp.len"].unique()[:5], "70%", round(len(dos)*.7))
print("time_delta =3e-6 share", (d["tcp.time_delta"]==3e-6).mean(), "<=3e-6", (d["tcp.time_delta"]<=3e-6).mean())
print(pd.Series([1,2,2,3,10]).skew(), pd.Series([1,2,2,3,10]).kurt())
print("conack contrib", (1428-62.5742)**2/62.5742)
```

04_cap9_firth.py

```
import sys, numpy as np, pandas as pd
d=pd.read_csv(sys.argv[1],low_memory=False,usecols=["tcp.len","mqtt.len","tcp.time_delta","mqtt.qos","mqtt.dupflag","target"])
y=(d.target!="legitimate").values.astype(float)
P=["tcp.len","mqtt.len","tcp.time_delta","mqtt.qos","mqtt.dupflag"]
X=np.column_stack([np.ones(len(d)),d[P].values])
def fit(X,y,fixed=None,it=200):
    b=np.zeros(X.shape[1]); free=np.ones(X.shape[1],bool)
    if fixed is not None: free[fixed]=False
    def pll(b):
        e=np.clip(X@b,-500,500); p=1/(1+np.exp(-e)); w=p*(1-p); I=(X*w[:,None]).T@X
        return np.sum(y*np.log(p+1e-300)+(1-y)*np.log(1-p+1e-300))+0.5*np.linalg.slogdet(I)[1],p,w,I
    for _ in range(it):
        L,p,w,I=pll(b); Ii=np.linalg.inv(I); h=w*np.einsum("ij,jk,ik->i",X,Ii,X)
        U=X.T@(y-p+h*(0.5-p)); Xf=X[:,free]; If=I[np.ix_(free,free)]
        step=np.linalg.solve(If,U[free]); s=1.0
        while True:
            bn=b.copy(); bn[free]+=s*step
            if pll(bn)[0]>=L-1e-9 or s<1e-4: break
            s/=2
        b=bn
        if np.max(np.abs(s*step))<1e-8: break
    L,p,w,I=pll(b); return b,np.sqrt(np.diag(np.linalg.inv(I))),L
b,se,L=fit(X,y); print("coef",np.round(b,4)); print("se",np.round(se,4))
for j,n in enumerate(P,1):
    _,_,L0=fit(X,y,fixed=j); print(n,"chisq",round(2*(L-L0),2))
```

05_cap8-9_cook_durbin_watson.py

```
import sys, numpy as np, pandas as pd, statsmodels.api as sm
from statsmodels.stats.stattools import durbin_watson
d=pd.read_csv(sys.argv[1],low_memory=False); y=(d.target!="legitimate").astype(int).values
X=sm.add_constant(d[["tcp.len","tcp.time_delta","mqtt.len"]]).values
g=sm.GLM(y,X,family=sm.families.Binomial()).fit(); p=np.clip(g.predict(),1e-12,1-1e-12); w=p*(1-p)
Ii=np.linalg.inv((X*w[:,None]).T@X); h=w*np.einsum("ij,jk,ik->i",X,Ii,X)
rp=(y-p)/np.sqrt(w); k=X.shape[1]; cook=rp**2*h/(k*(1-h)**2)
n=len(y); print("cook max",cook.max(),"argmax row",cook.argmax()+1,"n>4/n",(cook>4/n).sum(),(cook>4/n).mean
(),"n>0.5",(cook>0.5).sum(),"4/n",4/n)
r=g.resid_deviance; print("reziduuri deviance: max",np.abs(r).max(),">2",(np.abs(r)>2).sum(),">3",(np.abs(r)>
3).sum())
print("top5",np.sort(cook)[-5:]); np.save(sys.argv[2],cook)
m1=sm.OLS(d["mqtt.len"],sm.add_constant(d["tcp.len"])).fit(); m2=sm.OLS(d["tcp.time_delta"],sm.add_constant(d
[["tcp.len","mqtt.len"]])).fit()
print("DW1",durbin_watson(m1.resid),"DW2",durbin_watson(m2.resid))
ca=d["mqtt.conack.flags"].astype(str); print(pd.crosstab(ca,d["mqtt.msgtype"]))
print(pd.crosstab(d["mqtt.protoname"].astype(str),d["mqtt.msgtype"]))
m0=d[d["mqtt.msgtype"]==0]; print("msgtype0: hdrflags", m0["mqtt.hdrflags"].astype(str).unique()[:5], "mqtt.l
en max", m0["mqtt.len"].max())
print(pd.crosstab(d["mqtt.kalive"],d.target))
```

06_cap11_reguli_asociere.py

```
import sys, pandas as pd, numpy as np
from mlxtend.frequent_patterns import apriori, association_rules
d = pd.read_csv(sys.argv[1], low_memory=False)
F={"0x00000010":"ACK","0x00000018":"PSH+ACK","0x00000011":"FIN+ACK","0x00000002":"SYN","0x00000012":"SYN+ACK","0x00000004":"RST","0x00000019":"FIN+PSH+ACK","0x00000014":"RST+ACK"}
M={0:"fara",1:"CONNECT",2:"CONNACK",3:"PUBLISH",4:"PUBACK",5:"PUBREC",8:"SUBSCRIBE",9:"SUBACK",12:"PINGREQ",13:"PINGRESP",14:"DISCONNECT"}
t=pd.DataFrame({"flags":d["tcp.flags"].map(F),"msgtype":d["mqtt.msgtype"].astype(int).map(M),"qos":d["mqtt.qos"].astype(int),"dupflag":d["mqtt.dupflag"].astype(int),
 "len":pd.cut(d["tcp.len"],[-1,0,20,1460,1e9],labels=["0","1-20","21-1460","1461+"]),
 "delta":pd.cut(d["tcp.time_delta"],[-1,3e-6,2.6e-5,1.7e-4,1e9],labels=["Q1","Q2","Q3","Q4"]),"target":d.target})
print(t.head(5).to_string()); print(t.delta.value_counts(normalize=True).round(3).to_dict())
I=pd.get_dummies(t.astype(str),prefix_sep="="); print("items",I.shape[1])
fr=apriori(I,min_support=0.05,use_colnames=True); fr["k"]=fr.itemsets.apply(len); print("frequent per k",fr.k.value_counts().sort_index().to_dict())
r=association_rules(fr,metric="confidence",min_threshold=0.6); r=r[r.consequents.apply(len)==1]; print("rules",len(r))
def prune(r):
    keep=[]
    best={}
    for i,row in r.iterrows():
        c=next(iter(row.consequents)); a=row.antecedents
        sub=r[(r.consequents==row.consequents)&(r.antecedents.apply(lambda x: x<a))]
        keep.append(len(sub)==0 or row.confidence>sub.confidence.max())
    return r[keep]
rp=prune(r); rp=rp.assign(n=rp.antecedents.apply(len)+1); print("pruned",len(rp), rp.n.value_counts().sort_index().to_dict(), rp.support.min(),rp.support.max(),rp.lift.min(),rp.lift.max(),rp.lift.median())
print("RHS classes", rp[rp.consequents.apply(lambda x: next(iter(x)).startswith("target"))].consequents.apply(lambda x:next(iter(x))).value_counts().to_dict())
print(rp.sort_values("lift",ascending=False).head(10)[["antecedents","consequents","support","confidence","lift"]].to_string())
# per attack
for ms in [300,150]:
    fr2=apriori(I,min_support=ms/len(I),use_colnames=True,max_len=7)
    r2=association_rules(fr2,metric="confidence",min_threshold=0.5); r2=r2[r2.consequents.apply(len)==1]
    att=["target="+a for a in ["dos","bruteforce","malformed","slowite","flood"]]
    r2=r2[r2.consequents.apply(lambda x:next(iter(x)) in att) & r2.antecedents.apply(lambda x: not any(i.startswith("target") for i in x))]
    r2p=prune(r2); print(ms, r2p.consequents.apply(lambda x:next(iter(x))).value_counts().to_dict())
    if ms==150:
        if len(sys.argv)>2:  # opțional: regulile pe atacuri ca CSV (pentru graficul interactiv de pe site)
            pd.DataFrame({"stanga":r2p.antecedents.apply(lambda x:" & ".join(sorted(x))),"atac":r2p.consequents.apply(lambda x:next(iter(x))[7:]),
                          "pachete":(r2p.support*len(I)).round().astype(int),"support":r2p.support,"confidence":r2p.confidence,"lift":r2p.lift}).to_csv(sys.argv[2],index=False)
        for a in att:
            s=r2p[r2p.consequents.apply(lambda x:next(iter(x))==a)].sort_values(["confidence","support"],ascending=False).head(3)
            for _,x in s.iterrows(): print(a, sorted(x.antecedents), round(x.support*len(I)), round(x.support,4), round(x.confidence,3), round(x.lift,1))
```

07_grafice_corelatie.py

```
import sys, numpy as np, pandas as pd, matplotlib
matplotlib.use("Agg"); import matplotlib.pyplot as plt
from matplotlib.ticker import FuncFormatter
from scipy import stats
d=pd.read_csv(sys.argv[1],low_memory=False); out=sys.argv[2]
V=["tcp.time_delta","tcp.len","mqtt.len","mqtt.kalive","mqtt.qos","mqtt.dupflag","mqtt.retain","mqtt.conflag.cleansess","mqtt.conflag.passwd","mqtt.conflag.uname","mqtt.proto_len","mqtt.ver","mqtt.conack.val"]
ro=lambda v,dec=2: f"{(0.0 if round(v,dec)==0 else v):.{dec}f}".replace(".",",").replace("-","−")
cm=plt.get_cmap("RdBu").copy(); cm.set_bad("#dddddd")
def heat(g,title,fn):
    c=g[V].corr("spearman").values  # constant columns -> NaN (gray)
    fig,ax=plt.subplots(figsize=(11.7,9.88),dpi=100)
    im=ax.imshow(np.ma.masked_invalid(c),cmap=cm,vmin=-1,vmax=1)
    for i in range(len(V)):
        for j in range(len(V)):
            if not np.isnan(c[i,j]): ax.text(j,i,ro(c[i,j]),ha="center",va="center",fontsize=7.5,color="white" if abs(c[i,j])>0.5 else "black")
    ax.set_xticks(range(len(V))); ax.set_xticklabels(V,rotation=90,fontsize=9); ax.set_yticks(range(len(V))); ax.set_yticklabels(V,fontsize=9)
    cb=fig.colorbar(im,ax=ax,shrink=0.8); cb.ax.yaxis.set_major_formatter(FuncFormatter(lambda v,_: ro(v)))
    ax.set_title(title,fontsize=10); fig.tight_layout(); fig.savefig(f"{out}/{fn}"); plt.close(fig)
heat(d,"Corelația Spearman, variabile numerice, ordinale și binare, tot setul (gri = nu se poate calcula)","image40.png")
for k,cls in zip(range(42,48),["legitimate","dos","bruteforce","malformed","slowite","flood"]):
    heat(d[d.target==cls],f"Corelația Spearman, {cls} (gri = variabilă constantă în această clasă)",f"image{k}.png")
s=d.sample(8000,random_state=42)
fig,axs=plt.subplots(1,3,figsize=(15.6,4.94),dpi=100)
for ax,(x,y) in zip(axs,[("tcp.len","mqtt.len"),("tcp.time_delta","tcp.len"),("tcp.time_delta","mqtt.len")]):
    R,p=stats.spearmanr(d[x],d[y])  # R and p on all packets, plot on the sample
    rx,ry=s[x].rank(),s[y].rank(); ax.scatter(rx,ry,s=4,c="black",alpha=0.3)
    b1,b0=np.polyfit(rx,ry,1); xx=np.array([rx.min(),rx.max()]); ax.plot(xx,b0+b1*xx,color="#3355aa")
    ax.set_xlabel(f"{x} (rang)"); ax.set_ylabel(f"{y} (rang)"); ax.set_title(f"R (Spearman) = {ro(R)}, p < 0,001",fontsize=10)
    print(x,y,R,p)
fig.tight_layout(); fig.savefig(f"{out}/image41.png"); plt.close(fig)
```

08_grafice_restul.py

```
import sys, numpy as np, pandas as pd, matplotlib, statsmodels.api as sm
matplotlib.use("Agg"); import matplotlib.pyplot as plt
from matplotlib.ticker import FuncFormatter
from scipy import stats
from mlxtend.frequent_patterns import apriori, association_rules
d=pd.read_csv(sys.argv[1],low_memory=False); out=sys.argv[2]; cook=np.load(sys.argv[3])
d["is_attack"]=(d.target!="legitimate").astype(int)
CL=["legitimate","dos","bruteforce","malformed","slowite","flood"]
def g(v,_=None):
    s=("%g"%v).replace(".",",").replace("-","−"); return s
def ro(v,dec): s=f"{v:.{dec}f}"; return s.replace(".",",").replace("-","−")
def fix(ax,x=True):  # comma decimals on linear axes; x=False keeps category labels
    if x and ax.get_xscale()=="linear": ax.xaxis.set_major_formatter(FuncFormatter(g))
    if ax.get_yscale()=="linear": ax.yaxis.set_major_formatter(FuncFormatter(g))
def lab(v):  # 3.0 -> 3, hex/text unchanged
    if isinstance(v,float) and v.is_integer(): return f"{int(v):,}".replace(",",".")
    return str(v)
def save(fig,name,w,h):
    fig.set_size_inches(w/100,h/100); fig.savefig(f"{out}/{name}",dpi=100); plt.close(fig)
def stars(p): return "***" if p<0.001 else "**" if p<0.01 else "*" if p<0.05 else "ns"
# ---------- ch5 bar charts ----------
C5={2:"mqtt.conack.flags",3:"mqtt.conack.val",4:"mqtt.conflag.cleansess",5:"mqtt.conflag.passwd",6:"mqtt.conflag.uname",7:"mqtt.conflags",8:"mqtt.dupflag",9:"mqtt.hdrflags",12:"mqtt.msgtype",13:"mqtt.proto_len",14:"mqtt.protoname",15:"mqtt.qos",16:"mqtt.retain",17:"mqtt.ver",18:"tcp.flags"}
for k,c in C5.items():
    vc=d[c].value_counts(normalize=True)*100
    if len(vc)>10: vc=pd.concat([vc.iloc[:10],pd.Series({"Altele":vc.iloc[10:].sum()})])
    fig,ax=plt.subplots(); ax.bar([lab(v) for v in vc.index],vc.values); ax.set_title(c); ax.set_ylabel("% din pachete")
    plt.setp(ax.get_xticklabels(),rotation=45,ha="right"); fix(ax,x=False); fig.tight_layout(); save(fig,f"image{k}.png",600,420)
# ---------- ch6 ----------
C6={21:"tcp.flags",22:"tcp.time_delta",23:"tcp.len",24:"mqtt.hdrflags",25:"mqtt.msgtype",26:"mqtt.dupflag",27:"mqtt.qos",28:"mqtt.retain",29:"mqtt.len",30:"mqtt.conflags",31:"mqtt.conflag.cleansess",32:"mqtt.conflag.passwd",33:"mqtt.conflag.uname",34:"mqtt.kalive",35:"mqtt.protoname",36:"mqtt.proto_len",37:"mqtt.ver",38:"mqtt.conack.flags",39:"mqtt.conack.val"}
NUM=["tcp.time_delta","tcp.len","mqtt.len"]; KW_BARS=["mqtt.qos","mqtt.kalive"]
leg=d[d.target=="legitimate"]
def mw(c):
    r={}
    for a in CL[1:]:
        x=leg[c].values; y=d[d.target==a][c].values
        if np.unique(np.r_[x,y]).size==1: r[a]="-"
        else: r[a]=stars(stats.mannwhitneyu(x,y).pvalue)
    return r
for k,c in C6.items():
    fig,ax=plt.subplots()
    if c in NUM:
        data=[d[d.target==a][c].values for a in CL]
        if c=="tcp.time_delta": data=[np.where(v==0,1e-6,v) for v in data]; ax.set_yscale("log"); note="; 0 afișat ca 1e-06 s"
        else: ax.set_yscale("symlog",linthresh=1); note=""
        ax.boxplot(data,showfliers=False,widths=0.5); ax.set_xticks(range(1,7)); ax.set_xticklabels(CL)
        top=ax.get_ylim()[1]; s=mw(c)
        for i,a in enumerate(CL[1:],2): ax.text(i,top,s[a],ha="center",va="bottom")
        ax.set_ylim(top=top*(30 if c=="tcp.time_delta" else 8)); ax.set_title(f"{c}  (Kruskal-Wallis ***; stele: atac vs normal{note})",fontsize=9)
        print(c,s)
    else:
        s=d[c].map(lab) if d[c].dtype!=object else d[c].astype(str)
        top=s.value_counts().index[:6]; s=s.where(s.isin(top),"Altele")
        if c=="mqtt.kalive": s=d[c].map(lambda v:{0:"0",60:"60",65535:"65.535"}.get(v,"altele (1, 2, 3, 234)"))
        tab=pd.crosstab(s,d.target,normalize="columns")[CL]*100
        order=[v for v in sorted(tab.index,key=lambda v:(v.startswith(("Altele","altele")),len(v),v))]
        tab=tab.loc[order]; n=len(tab); wdt=0.8/n
        for i,(v,row) in enumerate(tab.iterrows()): ax.bar(np.arange(6)+(i-(n-1)/2)*wdt,row.values,wdt,label=v)
        ax.set_xticks(range(6)); ax.set_xticklabels(CL); ax.set_ylabel("% din clasă")
        if c in KW_BARS:
            st=mw(c); ax.set_ylim(0,112)
```

```
            for i,a in enumerate(CL[1:],1): ax.text(i,104,st[a],ha="center")
            ax.set_title(f"{c}  (Kruskal-Wallis ***; stele: atac vs normal)",fontsize=9); print(c,st)
        else: ax.set_title(f"{c}  (chi-pătrat ***)",fontsize=9)
        ax.legend(title="valoare",fontsize=7,title_fontsize=7,loc="upper left",bbox_to_anchor=(1.01,1))
    plt.setp(ax.get_xticklabels(),rotation=30,ha="right"); fix(ax,x=False); fig.set_size_inches(7.8,5.04); fig.tight_layout(); save(fig,f"image{k}.png",780,504)
# ---------- 7.4 Pearson / Spearman ----------
V=["tcp.len","mqtt.len","tcp.time_delta","mqtt.qos","mqtt.dupflag","is_attack"]
fig,axs=plt.subplots(1,2,figsize=(12.53,5.87))
for ax,m in zip(axs,["pearson","spearman"]):
    c=d[V].corr(m).values; im=ax.imshow(c,cmap="RdBu",vmin=-1,vmax=1)
    for i in range(6):
        for j in range(6): ax.text(j,i,ro(c[i,j],2),ha="center",va="center",fontsize=8,color="white" if abs(c[i,j])>0.5 else "black")
    ax.set_xticks(range(6)); ax.set_xticklabels(V,rotation=40,ha="right",fontsize=8); ax.set_yticks(range(6)); ax.set_yticklabels(V if m=="pearson" else [],fontsize=8); ax.set_title(m.capitalize())
cb=fig.colorbar(im,ax=axs,shrink=0.85); cb.ax.yaxis.set_major_formatter(FuncFormatter(lambda v,_:ro(v,2))); save(fig,"image48.png",1253,587)
# ---------- 8.1 / 8.2 / 8.3 ----------
smp=d.sample(8000,random_state=42)
m1=sm.OLS(d["mqtt.len"],sm.add_constant(d["tcp.len"])).fit(); L=np.log1p(d[["tcp.len","mqtt.len"]]); m1l=sm.OLS(L["mqtt.len"],sm.add_constant(L["tcp.len"])).fit()
fig,axs=plt.subplots(1,2)
axs[0].scatter(smp["tcp.len"],smp["mqtt.len"],s=4,alpha=0.4); xx=np.array([0,d["tcp.len"].max()]); axs[0].plot(xx,m1.params.iloc[0]+m1.params.iloc[1]*xx,"r")
axs[0].set_title(f"Valori brute, R² = {ro(m1.rsquared,3)}"); axs[0].set_xlabel("tcp.len"); axs[0].set_ylabel("mqtt.len")
ls=np.log1p(smp[["tcp.len","mqtt.len"]]); axs[1].scatter(ls["tcp.len"],ls["mqtt.len"],s=4,alpha=0.4); xx=np.array([0,L["tcp.len"].max()]); axs[1].plot(xx,m1l.params.iloc[0]+m1l.params.iloc[1]*xx,"r")
axs[1].set_title(f"După logaritm, R² = {ro(m1l.rsquared,3)}"); axs[1].set_xlabel("log(1 + tcp.len)"); axs[1].set_ylabel("log(1 + mqtt.len)")
for a in axs: fix(a)
fig.set_size_inches(14.3,5.46); fig.tight_layout(); save(fig,"image49.png",1430,546)
def rezid(a0,a1,fv,rs,t=""):  # reziduuri față de predicție + histograma cu curba normală (media și abaterea reziduurilor afișate)
    i=smp.index; a0.scatter(fv[i],rs[i],s=4,alpha=0.4); a0.axhline(0,color="r"); a0.set_title("Reziduuri față de predicție"+t); a0.set_xlabel("valoare prezisă"); a0.set_ylabel("reziduu")
    q=rs.quantile([.01,.99]); r=rs[(rs>=q.iloc[0])&(rs<=q.iloc[1])]; _,b,_=a1.hist(r,bins=50)
    xs=np.linspace(b[0],b[-1],300); a1.plot(xs,len(r)*(b[1]-b[0])*stats.norm.pdf(xs,r.mean(),r.std()),"r",lw=2,label="curba normală")
    a1.set_title("Histograma reziduurilor, 1% până la 99%"+t); a1.set_xlabel("reziduu"); a1.legend(frameon=False); fix(a0); fix(a1)
fig,axs=plt.subplots(2,2)
rezid(*axs[0],m1.fittedvalues,m1.resid," (brute)"); rezid(*axs[1],m1l.fittedvalues,m1l.resid," (logaritm)")
fig.set_size_inches(14.3,9.1); fig.tight_layout(); save(fig,"image50.png",1430,910)
m2=sm.OLS(d["tcp.time_delta"],sm.add_constant(d[["tcp.len","mqtt.len"]])).fit()
fig,axs=plt.subplots(1,2); rezid(*axs,m2.fittedvalues,m2.resid)
fig.set_size_inches(14.3,5.46); fig.tight_layout(); save(fig,"image51.png",1430,546)
fig,axs=plt.subplots(2,3)
for ax,a in zip(axs.flat,CL):
    gg=d[d.target==a]; m=sm.OLS(gg["mqtt.len"],sm.add_constant(gg["tcp.len"])).fit(); ss=gg if len(gg)<3000 else gg.sample(3000,random_state=42)
    ax.scatter(ss["tcp.len"],ss["mqtt.len"],s=4,alpha=0.4); xx=np.array([gg["tcp.len"].min(),gg["tcp.len"].max()]); ax.plot(xx,m.params.iloc[0]+m.params.iloc[1]*xx,"r")
    ax.set_title(f"{a}: panta {ro(m.params.iloc[1],3)}, R² = {ro(m.rsquared,3)}",fontsize=9); ax.set_xlabel("tcp.len",fontsize=8); ax.set_ylabel("mqtt.len",fontsize=8); ax.tick_params(labelsize=7); fix(ax)
fig.set_size_inches(16.9,9.1); fig.tight_layout(); save(fig,"image52.png",1690,910)
# ---------- 9: Cook, curve, probabilities ----------
fig,ax=plt.subplots(); ax.vlines(np.arange(1,len(cook)+1),0,cook,lw=0.5,alpha=0.6); n=len(cook)
ax.axhline(4/n,color="r",ls="--",lw=0.8,label=f"4/n = {ro(4/n*1e5,1)} · 10⁻⁵"); ax.set_title("Distanța Cook, modelul fără separare (maxim "+ro(cook.max(),4)+", pragul critic 0,5 sau 1)",fontsize=10)
ax.set_xlabel("numărul pachetului (rândul din fișier)"); ax.set_ylabel("distanța Cook"); ax.legend(fontsize=8); fix(ax)
ax.xaxis.set_major_formatter(FuncFormatter(lambda v,_: f"{int(v):,}".replace(",",".")))
fig.set_size_inches(13,5.2); fig.tight_layout(); save(fig,"image53.png",1300,520)
g6=sm.GLM(d.is_attack,sm.add_constant(d[["tcp.len","tcp.time_delta","mqtt.len"]]),family=sm.families.Binomial()).fit(); b=g6.params
t=np.linspace(0,2,200); pr=1/(1+np.exp(-(b.iloc[0]+b["tcp.len"]*12+b["tcp.time_delta"]*t+b["mqtt.len"]*10)))
fig,ax=plt.subplots(); ax.plot(t,pr,color="#d62728",lw=2); ax.axhline(0.5,color="gray",ls=":"); ax.set_ylim(0,1)
ax.set_title("Probabilitatea de atac prezisă de modelul final\n(tcp.len = 12 și mqtt.len = 10, valorile tipice; doar timpul variază)",fontsize=10); ax.set_xlabel("tcp.time_delta (secunde)"); ax.set_ylabel("P(atac)"); fix(ax)
fig.set_size_inches(10.5,4.8); fig.tight_layout(); save(fig,"image56.png",1050,480); print("curve p0",pr[0])
```

```
p=g6.predict(); fig,ax=plt.subplots(); bins=np.linspace(0,1,51)
ax.hist(p[d.is_attack==0],bins=bins,alpha=0.6,label="normal (0)"); ax.hist(p[d.is_attack==1],bins=bins,alpha=0.6,color="#e15759",label="atac (1)")
ax.set_yscale("log"); ax.set_xlabel("probabilitatea de atac prezisă"); ax.set_ylabel("pachete (scară log)"); ax.set_title("Ce scoate modelul final pentru fiecare pachet, separat pe grupul real",fontsize=10); ax.legend(); fix(ax)
fig.set_size_inches(10.5,4.8); fig.tight_layout(); save(fig,"image57.png",1050,480)
# ---------- 11.4 rules scatter ----------
F={"0x00000010":"ACK","0x00000018":"PSH+ACK","0x00000011":"FIN+ACK","0x00000002":"SYN","0x00000012":"SYN+ACK","0x00000004":"RST","0x00000019":"FIN+PSH+ACK","0x00000014":"RST+ACK"}
M={0:"fara",1:"CONNECT",2:"CONNACK",3:"PUBLISH",4:"PUBACK",5:"PUBREC",8:"SUBSCRIBE",9:"SUBACK",12:"PINGREQ",13:"PINGRESP",14:"DISCONNECT"}
tr=pd.DataFrame({"flags":d["tcp.flags"].map(F),"msgtype":d["mqtt.msgtype"].astype(int).map(M),"qos":d["mqtt.qos"].astype(int),"dupflag":d["mqtt.dupflag"].astype(int),
 "len":pd.cut(d["tcp.len"],[-1,0,20,1460,1e9],labels=["0","1-20","21-1460","1461+"]),"delta":pd.cut(d["tcp.time_delta"],[-1,3e-6,2.6e-5,1.7e-4,1e9],labels=["Q1","Q2","Q3","Q4"]),"target":d.target})
I=pd.get_dummies(tr.astype(str),prefix_sep="=")
def prune(r):
    keep=[]
    for _,row in r.iterrows():
        sub=r[(r.consequents==row.consequents)&(r.antecedents.apply(lambda x: x<row.antecedents))]
        keep.append(len(sub)==0 or row.confidence>sub.confidence.max())
    return r[keep]
r=association_rules(apriori(I,min_support=0.05,use_colnames=True),metric="confidence",min_threshold=0.6); r=prune(r[r.consequents.apply(len)==1])
r2=association_rules(apriori(I,min_support=150/len(I),use_colnames=True),metric="confidence",min_threshold=0.5); r2=r2[r2.consequents.apply(len)==1]
att=["target="+a for a in CL[1:]]; r2=prune(r2[r2.consequents.apply(lambda x:next(iter(x)) in att)&r2.antecedents.apply(lambda x:not any(i.startswith("target") for i in x))])
fig,axs=plt.subplots(1,2)
sc=axs[0].scatter(r.support,r.confidence,c=r.lift,cmap="Blues",s=14,edgecolors="gray",linewidths=0.3); cb=fig.colorbar(sc,ax=axs[0]); cb.set_label("lift"); cb.ax.yaxis.set_major_formatter(FuncFormatter(g))
axs[0].set_title(f"Rularea generală ({len(r)} reguli)"); axs[0].set_xlabel("support"); axs[0].set_ylabel("confidence")
cols={"dos":"#e8692f","bruteforce":"#1aae80","malformed":"#f0a30a","slowite":"#e377a6","flood":"#008000"}
for a in CL[1:]:
    s=r2[r2.consequents.apply(lambda x:next(iter(x))=="target="+a)]; axs[1].scatter(s.support,s.confidence,s=14,color=cols[a],label=a)
axs[1].set_xscale("log"); axs[1].set_title(f"Reguli pe atacuri ({len(r2)} reguli)"); axs[1].set_xlabel("support (scară logaritmică)"); axs[1].set_ylabel("confidence"); axs[1].legend(fontsize=8)
for a in axs: fix(a)
fig.set_size_inches(18,7.6); fig.tight_layout(); save(fig,"image64.png",1800,760)
```

09_cap10_arbori_ctree.R

```r
# Cap. 10: arborii de decizie cu partykit::ctree (test statistic + Bonferroni la fiecare nod).
# Rulare: Rscript 09_cap10_arbori_ctree.R ../mqttdataset_reduced.csv.zip folder_grafice
suppressMessages(library(partykit))
a <- commandArgs(TRUE)
d <- read.csv(unz(a[1], "mqttdataset_reduced.csv"), check.names = FALSE,
              colClasses = c(tcp.flags = "character", mqtt.conack.flags = "character",
                             mqtt.conflags = "character", mqtt.hdrflags = "character"))  # hex: text, nu num
ăr
names(d) <- make.names(names(d))
for (v in c(names(d)[sapply(d, is.character)], "mqtt.msgtype", "mqtt.dupflag", "mqtt.ver")) d[[v]] <- factor
(d[[v]])
options(digits = 10)  # pragurile din print, nerotunjite
d$is_attack <- factor(ifelse(d$target == "legitimate", "normal", "atac"), c("normal", "atac"))

# 70/30, separat pe fiecare clasă (aceleași proporții în ambele părți)
set.seed(42)
tr <- logical(nrow(d))
for (cl in levels(d$target)) { i <- which(d$target == cl); tr[sample(i, round(0.7 * length(i)))] <- TRUE }
D <- d[tr, ]; Te <- d[!tr, ]
cat("constructie", nrow(D), "test", nrow(Te), "\n"); print(table(D$target)); print(table(D$is_attack))

ctl <- function(md, tt = "Bonferroni") ctree_control(teststat = "quadratic", testtype = tt, mincriterion =
0.95,
                                                     minsplit = 20, minbucket = 300, maxdepth = md)
X7 <- c("tcp.flags", "mqtt.msgtype", "tcp.len", "tcp.time_delta", "mqtt.qos", "mqtt.dupflag", "mqtt.ver")
f_cls <- reformulate(X7, "is_attack")
f_reg <- tcp.len ~ target + tcp.flags + mqtt.msgtype + tcp.time_delta

met <- function(y, p) { tp <- sum(p == "atac" & y == "atac"); fp <- sum(p == "atac" & y == "normal")
  fn <- sum(p == "normal" & y == "atac"); s <- tp / (tp + fn); pr <- tp / (tp + fp)
  round(c(acc = mean(p == y), sens = s, prec = pr, f1 = 2 * s * pr / (s + pr)), 4) }
reg <- function(y, p) round(c(MAE = mean(abs(y - p)), RMSE = sqrt(mean((y - p)^2)),
                              R2 = 1 - sum((y - p)^2) / sum((y - mean(y))^2)), 4)

out <- if (is.na(a[2])) NULL else { dir.create(a[2], showWarnings = FALSE); a[2] }
salv <- function(x, f) { print(x); if (!is.null(out)) write.csv(x, file.path(out, f), row.names = FALSE) }

cat("\n10.3 maxdepth, clasificare\n")
salv(do.call(rbind, lapply(2:6, function(md) { m <- ctree(f_cls, D, control = ctl(md))
  data.frame(maxdepth = md, frunze = width(m), t(met(D$is_attack, predict(m))[c(1, 4)]), t(met(Te$is_attack,
predict(m, Te))[c(1, 4)])) })),
  "adancime_clasificare.csv")  # coloane: acc, f1 constructie; acc.1, f1.1 test
cat("\n10.3 maxdepth, regresie\n")
salv(do.call(rbind, lapply(2:6, function(md) { m <- ctree(f_reg, D, control = ctl(md))
  data.frame(maxdepth = md, frunze = width(m), R2_constr = reg(D$tcp.len, predict(m))[[3]], t(reg(Te$tcp.le
n, predict(m, Te)))) })),
  "adancime_regresie.csv")

m <- ctree(f_cls, D, control = ctl(5))
cat("\n10.4 arborele de clasificare\n"); print(m)
cat("aceleasi predictii la maxdepth 6:", identical(predict(m, Te), predict(ctree(f_cls, D, control = ctl
(6)), Te)), "\n")

cat("\n10.1 radacina: statistica, df, log10 p, log10 p*7\n")
st <- info_node(node_party(m))$criterion["statistic", ]
df <- sapply(X7, function(v) if (is.factor(D[[v]])) nlevels(droplevels(D[[v]])) - 1 else 1)
lp <- pchisq(st, df, lower.tail = FALSE, log.p = TRUE) / log(10)
print(round(cbind(T = st, df = df, log10p = lp, log10p7 = lp + log10(7))[order(lp), ], 2))
q <- table(D$mqtt.qos > 0, D$is_attack); E <- outer(rowSums(q), colSums(q)) / sum(q)
cat("tabel qos: 0\n"); print(q); cat("E\n"); print(round(E, 1)); cat("chi2", sum((q - E)^2 / E), " (n-1) r
2", (nrow(D) - 1) * cor(D$mqtt.qos, as.numeric(D$is_attack))^2, " r", cor(D$mqtt.qos, as.numeric(D$is_attack)),
"\n")

mu <- ctree(f_cls, D, control = ctl(5, "Univariate"))
cat("\nfara Bonferroni: frunze", width(mu), " aceleasi predictii pe test:", identical(predict(mu, Te), predi
ct(m, Te)), "\n")

cat("\nfrunze: id | regula | n | % atac | prezice | clase\n")
fn <- predict(m, type = "node"); rl <- partykit:::.list.rules.party(m)
for (id in sort(unique(fn))) { i <- fn == id; k <- sort(table(D$target[i]), decreasing = TRUE); k <- k[k >
0]
```

```
  cat(id, "|", rl[as.character(id)], "|", sum(i), "|", round(100 * mean(D$is_attack[i] == "atac"), 2), "|",
      as.character(predict(m, D[which(i)[1], ])), "|", paste(names(k), k, collapse = "; "), "\n") }

p <- predict(m, Te)
cat("\n10.5 test\n"); print(table(real = Te$is_attack, prezis = p))
cat("test", met(Te$is_attack, p), "\nconstructie", met(D$is_attack, predict(m)), "\n")
print(t(sapply(split(p != Te$is_attack, Te$target), function(x) c(n = length(x), gresite = sum(x)))))
ratate <- Te$is_attack == "atac" & p == "normal"
cat("atacuri ratate pe frunze:\n"); print(table(predict(m, Te[ratate, ], type = "node")))

cat("\nverificari 10.2 / 10.4\n")
X19 <- setdiff(names(d)[sapply(d, function(x) length(unique(x)) > 1)], c("target", "is_attack", "mqtt.msg",
"mqtt.msgid"))
m19 <- ctree(reformulate(X19, "is_attack"), D, control = ctl(5))
cat(length(X19), "variabile: F1 test", met(Te$is_attack, predict(m19, Te))[4], " radacina:", names(m19$data)
[split_node(node_party(m19))$varid], "\n")
D$len3 <- cut(D$tcp.len, c(-1, 0, 20, Inf)); Te$len3 <- cut(Te$tcp.len, c(-1, 0, 20, Inf))
mb <- ctree(reformulate(c(setdiff(X7, "tcp.len"), "len3"), "is_attack"), D, control = ctl(5))
cat("tcp.len in 0 / 1-20 / 21+: F1 test", met(Te$is_attack, predict(mb, Te))[4], "\n")

r <- ctree(f_reg, D, control = ctl(4))
cat("\n10.6 arborele de regresie\n"); print(r)
pr <- predict(r, Te); e <- Te$tcp.len - pr
cat("constructie", reg(D$tcp.len, predict(r)), "\ntest", reg(Te$tcp.len, pr), "\n")
cat("suma |e|", sum(abs(e)), " SSE", sum(e^2), " SST", sum((Te$tcp.len - mean(Te$tcp.len))^2), "\n")
cat("cota SSE pe clase:\n"); print(round(tapply(e^2, Te$target, sum) / sum(e^2), 4))
rn <- predict(r, type = "node")
cat("frunze regresie: id | n | media | mediana | clase\n")
for (id in sort(unique(rn))) { i <- rn == id; k <- table(droplevels(D$target[i]))
  cat(id, "|", sum(i), "|", round(mean(D$tcp.len[i]), 1), "|", median(D$tcp.len[i]), "|", paste(names(k), k,
collapse = "; "), "\n") }

cat("\ncomparatie cu regresia liniara (R2 test: dreapta, arbore)\n")
for (f in list(mqtt.len ~ tcp.len, tcp.time_delta ~ tcp.len + mqtt.len)) {
  l <- lm(f, D); t <- ctree(f, D, control = ctl(4)); y <- Te[[all.vars(f)[1]]]
  cat(deparse(f), reg(y, predict(l, Te))[3], reg(y, predict(t, Te))[3], "\n") }

# nodurile, pentru desene (10_grafice_arbori.py): id, copii, variabila, p ajustat, etichetele ramurilor, fru
nze
noduri <- function(t, y) {
  fn <- predict(t, type = "node")
  do.call(rbind, lapply(nodeids(t), function(id) { nd <- nodeapply(t, id, identity)[[1]]; sp <- split_node(n
d)  # id-urile din arborele întreg
    v <- y[fn %in% nodeids(t, id, terminal = TRUE)]
    r <- data.frame(id = id, copii = "", var = "", p = NA, ramuri = "", n = length(v))
    if (!is.null(sp)) { r$copii <- paste(sapply(kids_node(nd), id_node), collapse = ";"); r$var <- names(t$d
ata)[sp$varid]
      r$p <- info_node(nd)$p.value; r$ramuri <- paste(character_split(sp, t$data, digits = 6)$levels, collap
se = "|") }
    cbind(r, if (is.factor(y)) t(as.matrix(table(v))) else t(c(quantile(v, c(0, .25, .5, .75, 1)), media = m
ean(v)))) }))
}
if (!is.null(out)) {
  write.csv(noduri(m, D$target), file.path(out, "noduri_clasificare.csv"), row.names = FALSE)
  write.csv(noduri(r, D$tcp.len), file.path(out, "noduri_regresie.csv"), row.names = FALSE)
}
```

10_grafice_arbori.py

```python
# Graficele din cap. 10, din CSV-urile scrise de 09_cap10_arbori_ctree.R.
# Rulare: python 10_grafice_arbori.py folder_csv folder_iesire
import sys, re, textwrap, numpy as np, pandas as pd, matplotlib
matplotlib.use("Agg"); import matplotlib.pyplot as plt
from matplotlib.patches import Rectangle, Patch
from matplotlib.lines import Line2D

src, out = sys.argv[1], sys.argv[2]
CL = {"legitimate": "#2a7ad6", "dos": "#e8692f", "bruteforce": "#1aae80", "malformed": "#f0a30a", "slowite": "#e377a6", "flood": "#008000"}
ro = lambda v, dec: f"{v:,.{dec}f}".replace(",", " ").replace(".", ",").replace(" ", ".")
g = lambda v: ("%g" % v).replace(".", ",")

def ramura(s):  # "<= 1e-06" -> "≤ 1e-06"; "0x00000018" -> "0x18"; lista -> "∈ {...}"
    s = re.sub(r"0x0+([0-9a-f]{2})", r"0x\1", s)
    m = re.match(r"(<=|>) (.+)", s)
    if m: return ("≤ " if m[1] == "<=" else "> ") + g(float(m[2]))
    v = s.split(", ")
    return "= " + v[0] if len(v) == 1 else textwrap.fill("∈ {" + ", ".join(v) + "}", 16)

def arbore(csv, titlu, fisier, frunza, legenda):
    t = pd.read_csv(csv, keep_default_na=False).set_index("id")
    kids = {i: [int(k) for k in str(r.copii).split(";")] if r.copii else [] for i, r in t.iterrows()}
    x, y, frunze = {}, {}, []
    def pozitie(i, dep):
        y[i] = -dep
        if not kids[i]: x[i] = len(frunze); frunze.append(i)
        else:
            for k in kids[i]: pozitie(k, dep + 1)
            x[i] = np.mean([x[k] for k in kids[i]])
    pozitie(1, 0)
    jos = min(y.values()) - 0.6
    fig, ax = plt.subplots(figsize=(17, 12))
    for i in t.index:  # liniile și etichetele ramurilor
        if not kids[i]: continue
        r = t.loc[i]
        for k, lab in zip(kids[i], r.ramuri.split("|")):
            yk = jos if not kids[k] else y[k]
            ye = yk + (0.45 if not kids[k] else 0)
            ax.plot([x[i], x[k]], [y[i], ye], color="#999", lw=1, zorder=1)
            f = 0.8 if not kids[k] else 0.55  # eticheta mai aproape de copil, ca să nu se suprapună cu a fratelui
            ax.text(x[i] + f * (x[k] - x[i]), y[i] + f * (ye - y[i]), ramura(lab), ha="center", va="center", fontsize=9,
                    bbox=dict(fc="white", ec="none", pad=0.5), zorder=2)
        p = float(r.p)
        ax.text(x[i], y[i], f"{i}  {r['var']}\n" + ("p < 0,001" if p < 0.001 else "p = " + ro(p, 3)), ha="center", va="center",
                fontsize=10, bbox=dict(boxstyle="round", fc="#eef3fb", ec="#7f9fd4"), zorder=3)
    for i in frunze:
        ax.text(x[i], jos + 0.2, str(i), ha="center", fontsize=10)
        frunza(ax, x[i], jos, t.loc[i])
    ax.set_xlim(-0.8, len(frunze) - 0.2); ax.set_ylim(jos - 2.4, 0.5); ax.axis("off")
    ax.set_title(titlu, fontsize=13)
    ax.legend(handles=legenda, loc="lower center", ncol=len(legenda), frameon=False, bbox_to_anchor=(0.5, -0.02), fontsize=10)
    fig.tight_layout(); fig.savefig(f"{out}/{fisier}", dpi=100); plt.close(fig)

def bara(ax, xi, jos, r):  # bara din frunză: din ce clase sunt pachetele
    b = jos - 1.75
    for c, col in CL.items():
        h = 1.6 * r[c] / r.n
        ax.add_patch(Rectangle((xi - 0.25, b), 0.5, h, fc=col, ec="white", lw=0.8)); b += h
    atac = 1 - r.legitimate / r.n
    ax.text(xi, jos - 2.0, f"n = {ro(r.n, 0)}\natac {ro(100 * atac, 1)}%", ha="center", va="top", fontsize=9)

H = lambda v: 1.6 * np.log10(1 + v) / np.log10(1 + 32768)  # scară logaritmică, până la maximul tcp.len
def cutie(ax, xi, jos, r):  # boxplot tcp.len: Q1-Q3, mediana, media (valoarea prezisă)
    b = jos - 1.75
    ax.plot([xi, xi], [b + H(r["0%"]), b + H(r["100%"])], color="#444", lw=1)
    ax.add_patch(Rectangle((xi - 0.2, b + H(r["25%"])), 0.4, max(H(r["75%"]) - H(r["25%"]), 0.004), fc="#cfe2
```

```
fb", ec="#2a7ad6"))
    ax.plot([xi - 0.2, xi + 0.2], [b + H(r["50%"])] * 2, color="#123f80", lw=2.5)
    ax.plot(xi, b + H(r.media), "D", color="#e8692f", ms=6)
    ax.text(xi, jos - 2.0, f"n = {ro(r.n, 0)}\nmedia {ro(r.media, 1)}", ha="center", va="top", fontsize=9)
    if xi == 0:
        for v in [0, 10, 100, 1000, 10000]:
            ax.text(-0.55, b + H(v), ro(v, 0), ha="right", va="center", fontsize=8, color="#666")
            ax.plot([-0.52, -0.47], [b + H(v)] * 2, color="#666", lw=1)
        ax.plot([-0.5, -0.5], [b, b + 1.6], color="#666", lw=1); ax.text(-0.5, b + 1.7, "tcp.len", ha="cente
r", fontsize=8, color="#666")

arbore(f"{src}/noduri_clasificare.csv", "Arborele de clasificare (is_attack), maxdepth = 5", "arbore_clasific
are.png", bara,
       [Patch(fc=c, label=k) for k, c in CL.items()])
arbore(f"{src}/noduri_regresie.csv", "Arborele de regresie (tcp.len), maxdepth = 4", "arbore_regresie.png", c
utie,
       [Patch(fc="#cfe2fb", ec="#2a7ad6", label="Q1 până la Q3"), Line2D([], [], color="#123f80", lw=2.5, lab
el="mediana"),
        Line2D([], [], marker="D", color="#e8692f", ls="", label="media (valoarea prezisă)")])

# alegerea adâncimii (10.3)
c = pd.read_csv(f"{src}/adancime_clasificare.csv"); r = pd.read_csv(f"{src}/adancime_regresie.csv")
fig, axs = plt.subplots(1, 2, figsize=(15.3, 5.78))
for ax, (tr, te, nume, ales) in zip(axs, [(c.f1, c["f1.1"], "Arborele de clasificare: F1", 5), (r.R2_constr,
r.R2, "Arborele de regresie: R²", 4)]):
    ax.plot(c.maxdepth, tr, "o-", color="#2a7ad6", lw=2.5, ms=8, label="construcție (70%)")
    ax.plot(c.maxdepth, te, "o--", color="#e8692f", lw=2.5, ms=8, label="test (30%)")
    for md, v in zip(c.maxdepth, te): ax.annotate(ro(v, 3), (md, v), textcoords="offset points", xytext=(0, -
16), ha="center", fontsize=9)
    ax.axvline(ales, color="gray", ls=":"); ax.text(ales + 0.08, ax.get_ylim()[0], f"ales: {ales}", color="gr
ay", va="bottom")
    ax.set_title(nume); ax.set_xlabel("maxdepth"); ax.set_xticks(c.maxdepth); ax.legend(frameon=False, loc="u
pper left")
    ax.yaxis.set_major_formatter(lambda v, _: g(round(v, 3))); ax.grid(axis="y", alpha=0.3)
    ax.spines[["top", "right"]].set_visible(False)
fig.tight_layout(); fig.savefig(f"{out}/adancime.png", dpi=100); plt.close(fig)
```